

Lab 1 Google Vertex AI Agent Development Kit

Objectives

In this lab, you will create a single agent that can use a search tool. You will test agents in ADK's browser UI, from a CLI chat interface, and programmatically from within a script.

Task 1. Install ADK and set up your environment

Prepare a Cloud Shell Editor tab

1. With your Google Cloud console window selected, open Cloud Shell by pressing the **G** key and then the **S** key on your keyboard. Alternatively, you can click the Activate Cloud Shell button () in the upper right of the Cloud console.
2. Click **Continue**.
3. When prompted to authorize Cloud Shell, click **Authorize**.
4. In the upper right corner of the Cloud Shell Terminal panel, click the **Open in new window** button  .
5. In the Cloud Shell Terminal, enter the following to open the Cloud Shell Editor to your home directory:
cloudshell workspace ~
6. Close any additional tutorial or Gemini panels that appear on the right side of the screen to save more of your window for your code editor.
7. Throughout the rest of this lab, you can work in this window as your IDE with the Cloud Shell Editor and Cloud Shell Terminal.

Download and install the ADK and code samples for this lab

1. Update your PATH environment variable and **install ADK** by running the following commands in the Cloud Shell Terminal. *Note: You will specify the version to ensure that the version of ADK that you install corresponds to the version used in this lab:*

```
export PATH=$PATH:"/home/${USER}/.local/bin"
```

```
python3 -m pip install google-adk
```

2. Paste the following commands into the Cloud Shell Terminal to copy a file from a Cloud Storage bucket, and unzip it, creating a project directory with code for this lab:

```
gcloud storage cp gs://ags-genai-demos/adk\_project.zip ./adk_project.zip
```

```
unzip adk_project.zip
```

Install additional lab requirements with:

```
python3 -m pip install -r adk_project/requirements.txt
```

Core Concepts of Agent Development Kit

Google ADK is built around a few core concepts that make it powerful and flexible:

- **Agent:** Agents are core building blocks designed to accomplish specific tasks. They can be powered by LLMs to reason, plan, and utilize tools to achieve goals, and can even collaborate on complex projects.
- **Tools:** Tools give agents abilities beyond conversation, letting them interact with external APIs, search information, run code, or call other services.
- **Session Services:** Session services handle the context of a single conversation (Session), including its history (Events) and the agent's working memory for that conversation (State).
- **Callbacks:** Custom code snippets you provide to run at specific points in the agent's process, allowing for checks, logging, or behavior modifications.
- **Artifact Management:** Artifacts allow agents to save, load, and manage files or binary data (like images or PDFs) associated with a session or user.
- **Runner:** The engine that manages the execution flow, orchestrates agent interactions based on Events, and coordinates with backend services.

Task 2. Review the structure of Agent Development Kit project directories

1. In the Cloud Shell Editor's file explorer pane, find the **adk_project** folder. Click it to toggle it open.
2. This directory contains three other directories: **my_google_search_agent**, **app_agent**, and **llm_auditor**. Each of these directories represents a separate agent. Separating agents into their own directories within a project directory provides organization and allows Agent Development Kit to understand what agents are present.
3. Click on the **my_google_search_agent** to explore an agent directory.
4. Notice that the directory contains an `__init__.py` file and an `agent.py` file. An `__init__.py` file is typically used to identify a directory as a Python package that can be imported by other Python code. **Click the `init.py` file** to view its contents.

5. Notice that the `__init__.py` file contains a single line, which imports from the `agent.py` file. ADK uses this to identify this directory as an agent package:

`from . import agent`

6. Now click on the `agent.py` file. This file consists of a simple agent. You will equip it with a powerful tool: the ability to search the internet using Google Search. Notice a few things about the file:

- Notice the imports from `google.adk`: the `Agent` class and the `google_search` tool from the `tools` module
- Read the code comments that describe the parameters that configure this simple agent.

7. To use the imported `google_search` tool, it needs to be passed to the agent. Do that by **pasting the following line** into the `agent.py` file where indicated at the end of the `Agent` object creation:

```
tools=[google_search]
```

8. **Save** the file.

Tools enable an agent to perform actions beyond generating text. In this case, the `google_search` tool allows the agent to decide when it would like more information than it already has from its training data. It can then write a search query, use Google Search to search the web, and then base its response to the user on the results. When a model bases its response on additional information that it retrieves, it is called "grounding," and this overall process is known as "retrieval-augmented generation" or "RAG."

You can learn more about how to use tools with ADK in the lab *Empower ADK agents with tools*.

Task 3. Run the agent using the ADK's Dev UI

ADK includes a development UI designed to run locally to help you develop and test your agents. It can help you visualize what each agent is doing and how multiple agents interact with one another. You will explore this interface in this task.

When you run an agent, the ADK needs to know who is requesting the model API calls. You can provide this information in one of two ways. You can:

1. Provide a [Gemini API key](#).
2. Authenticate your environment with Google Cloud credentials and associate your model API calls with a Vertex AI project and location.

In this lab, you will take the Vertex AI approach.

1. In the Cloud Shell Editor menus, select **View > Toggle Hidden Files** to view or hide your hidden files (files with a period at the start of their filename are hidden by default in most file systems). You may need to scroll down in this menu to find the Toggle Hidden Files option.
2. In the Cloud Shell Editor file explorer pane, navigate to the **adk_project/my_google_search_agent** directory.
3. Select the **.env** file in the **my_google_search_agent** directory.
4. Paste these values over what is currently in the file to **update the file to include your project ID**:

GOOGLE_GENAI_USE_VERTEXAI=TRUE

GOOGLE_CLOUD_PROJECT= gen-ai-training-476514

GOOGLE_CLOUD_LOCATION=us-central1

MODEL=gemini-2.5-flash

5. **Save** the file.

When you test your agent using ADK's dev UI or the command-line chat interface, they will load and use an agent's `.env` file if one is present or else look for environment variables with the same names as those set here.

6. In the Cloud Shell Terminal, ensure you are in the **adk_project** directory where your agent subdirectories are located by running:

```
cd ~/adk_project
```

Copied!

7. **Launch the Agent Development Kit Dev UI** with the following command:

```
adk web
```

Output

```
INFO: Started server process [2434]
```

```
INFO: Waiting for application startup.
```

```
+-----+
```

```
| ADK Web Server started |
```

```
| |
```

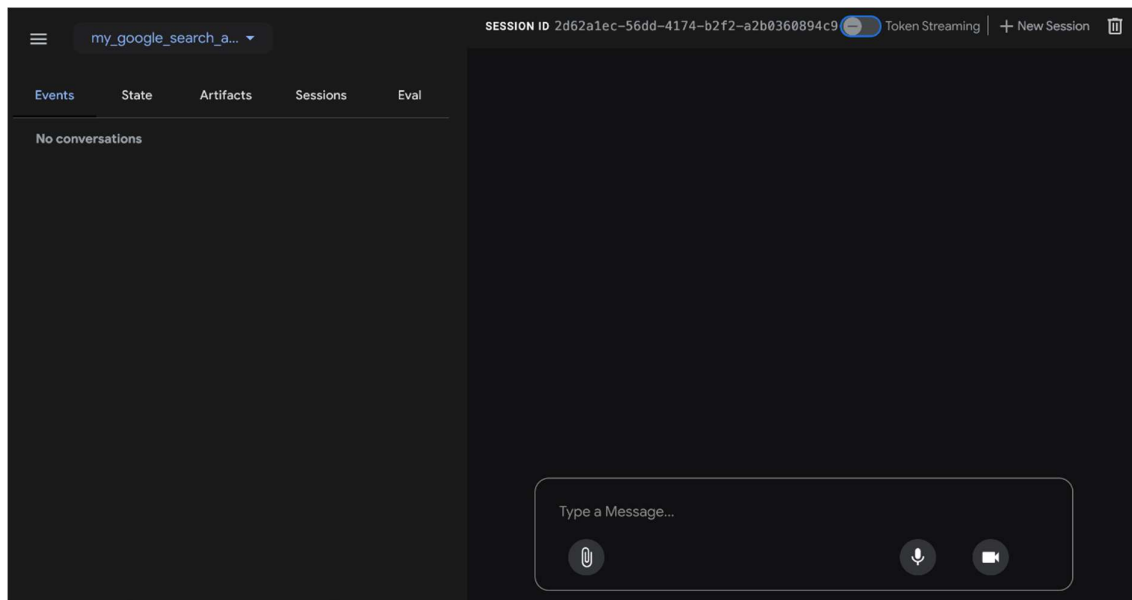
```
| For local testing, access at http://localhost:8000. |
```

+-----+

INFO: Application startup complete.

INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

8. To view the web interface in a new tab, click the **http://127.0.0.1:8000** link in the Terminal output, which will link you via proxy to this app running locally on your Cloud Shell instance.
9. A new browser tab will open with the ADK Dev UI.
10. From the **Select an agent** dropdown on the left, select **my_google_search_agent**.



11. In order to encourage the agent to use its Google Search tool, enter the question:
What is some recent global news?

12. You will notice from the results that the agent is able to use Google Search to get up-to-date information, rather than having its information stop on the date when its model was trained.

A response using grounding with Google search includes ready-to-display HTML "Search Suggestions" like those you see at the bottom of the agent's response. When you use grounding with Google Search, you are [required to display these suggestions](#), which help users follow up on the information the model used for its response.

13. Notice that in the left side bar, you are in the **Trace** tab by default. Click on your last query text (What is some recent global news?) to see a trace of how long

different parts of your query took to execute. You can use this to debug more complex executions involving tool calls to understand how various processes contribute to the latency of your responses.



The screenshot shows a dark-themed interface. At the top, a light gray box contains the query "What is some recent global news?". Below this, the "Invocation ID" is displayed as "e-fc714684-1a71-4f1f-b969-ea8690f89ff9". A list of execution steps follows, each with an icon, a label, and a duration in a blue bar:

- A right-pointing arrow icon followed by "invocation" and a duration of 4963.50ms.
- A person walking icon followed by "agent_run [google_sear..." and a duration of 4962.82ms.
- A speech bubble icon followed by "call_llm" and a duration of 4847.93ms.

14. Click the agent icon () next to the agent's response (or an event from the list on the **Events** tab) to inspect the event returned by the agent, which includes the content returned to the user and groundingMetadata which details the search results that the response was based on.
15. When you are finished exploring the dev UI, close this browser tab and return to your browser tab with the Cloud Shell Terminal, click on the terminal's pane, and press CTRL + C to stop the web server.

Click **Check my progress** to verify the objective.

Run the agent using the ADK's Web UI.

Task 4. Run an agent programmatically

While the dev UI is great for testing and debugging, it is not suitable for presenting your agent to multiple users in production.

To run an agent as part of a larger application, you will need to include a few additional components in your **agent.py** script that the web app handled for you in the previous task. Proceed with the following steps to open a script with these components to review them.

1. In the Cloud Shell Terminal run the following commands to export environment variables. You can use this approach to set environment variables for all of your agents to use if they do not have a .env file in their directory:

```
export GOOGLE_GENAI_USE_VERTEXAI=TRUE
```

```
export GOOGLE_CLOUD_PROJECT= gen-ai-training-476514
```

```
export GOOGLE_CLOUD_LOCATION=us-central1
```

```
export MODEL=gemini-2.5-flash
```

2. In the Cloud Shell Editor file browser, select the **adk_project/app_agent** directory.
3. Select the **agent.py** file in this directory.
4. This agent is designed to run as part of an application. Read the commented code in **agent.py**, paying particular attention to the following components in the code:

Component	Feature	Description
InMemoryRunner()	Oversight of agent execution	The Runner is the code responsible for receiving the user's query, passing it to the appropriate agent, receiving the agent's response event and passing it back to the calling application or UI, and then triggering the following event. You can read more in the ADK documentation about the event loop .
runner.session_service.create_session()	Conversation history & shared state	Sessions allow an agent to preserve state, remembering a list of items, the current status of a task, or other 'current' information. This class creates a local session service for simplicity, but in production this could be handled by a database.

types.Content() and types.Part()	Structured, multimodal messages	Instead of a simple string, the agent is passed a Content object which can consist of multiple Parts. This allows for complex messages, including text and multimodal content to be passed to the agent in a specific order.
----------------------------------	---------------------------------	--

When you ran the agent in the dev UI, it created a session service, artifact service, and runner for you. When you write your own agents to deploy programmatically, it is recommended that you provide these components as external services rather than relying on in-memory versions.

5. Notice that the script includes a hardcoded query, which asks the agent: "What is the capital of France?"
6. Run the following command in the Cloud Shell Terminal to run this agent programmatically:

```
python3 app_agent/agent.py
```

Selected Output:

trivia_agent: The capital of France is Paris.

7. You can also define specific input and/or output schema for an agent.

Task 5. Chat with an agent via the command-line interface

You can also chat with an agent in your local development environment by using the command line interface. This can be very handy for quickly debugging and testing agents as you develop them.

Like the web interface, the command line interface also handles the creation of the session service, artifact service, and runner for your agent.

To run an interactive session using the command line interface:

1. **Run** the following in Cloud Shell Terminal:

```
adk run my_google_search_agent
```

Output:

Log setup complete: /tmp/agents_log/agent.20250322_010300.log

To access latest log: tail -F /tmp/agents_log/agent.latest.log

Running agent basic_search_agent, type exit to exit.

user:

Input the following message:

what are some new movies that have been released in the past month in India?

Example output (yours may be a little different):

[google_search_agent]: Here are some movies that have been released in India in the past month (August-September 2025):

- * **Param Sundari** Released on 29 August 2025.
- * **Ek Chatur Naar** Released on 12 September 2025.
- * **Jolly LLB 3** Released on 19 September 2025.
- * **Ajey: The Untold Story of a Yogi** Released on 19 September 2025.
- * **Antha 7 Naatkal** Released on Sep 25 2025.

When you are finished chatting with the command line interface, **enter exit** at the next user prompt to end the chat.

ADDITIONAL LABS AND EXERCISES

Run the ADK based agent locally in VS Code

Explore how you can run agents locally on your laptop by installing Google ADK Python libraries

Try out the Agent demos from Google ADK Agents library

<https://github.com/google/adk-samples/tree/main/python/agents/medical-pre-authorization>

<https://github.com/google/adk-samples/tree/main/python/agents/>

